

OS SKILL 11

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define HISTORY_SIZE 5
#define CMD_LEN 100

char history[HISTORY_SIZE][CMD_LEN];
int history_count = 0;

void add_history(const char *cmd) {
    if (history_count < HISTORY_SIZE) {
        strcpy(history[history_count], cmd);
        history_count++;
    } else {
        for (int i = 1; i < HISTORY_SIZE; i++) {
            strcpy(history[i - 1], history[i]);
        }

        strcpy(history[HISTORY_SIZE - 1], cmd);
    }
}

void display_history() {
    printf("=== Command History ===\n");

    for (int i = 0; i < history_count; i++) {
        printf("%d. %s\n", i + 1, history[i]);
    }
}

void retrieve_entry(int index) {
    if (index >= 1 && index <= history_count)
        printf("Retrieved: %s\n", history[index - 1]);
    else
        printf("Invalid history index\n");
}
```

```
void run_pipeline() {
    int pipefd[2];

    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(1);
    }

    pid_t pid1 = fork();

    if (pid1 == 0) {
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    pid_t pid2 = fork();

    if (pid2 == 0) {
        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[1]);
        close(pipefd[0]);

        execlp("wc", "wc", "-l", NULL);

        perror("execlp wc");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Pipeline execution completed\n");
}
```

```

int main() {
    printf("=== Store Commands ===\n");

    add_history("pwd");
    add_history("ls");
    add_history("echo Hello");
    add_history("date");
    add_history("whoami");

    display_history();

    printf("\n=== Retrieve History Entry ===\n");
    retrieve_entry(3);

    printf("\n=== Capacity Limit Test ===\n");

    add_history("uname -a");

    display_history();

    printf("\n=== History Consistency Test ===\n");

    if (history_count <= HISTORY_SIZE)
        printf("History buffer size maintained correctly\n");

    printf("\n=== Pipeline Structure ===\n");
    printf("Command 1: ls -l\n");
    printf("Command 2: wc -l\n");

    printf("\n=== Pipeline Execution ===\n");

    run_pipeline();

    printf("\n=== Pipeline Layout Validation ===\n");
    printf("ls -l | wc -l\n");
    printf("Execution order maintained correctly\n");

    printf("\n=== Task 11 Complete ===\n");

    return 0;
}

```

Outputs:

```
=== Store Commands ===  
=== Command History ===  
1. pwd  
2. ls  
3. echo Hello  
4. date  
5. whoami
```

```
5. whoami  
  
=== Retrieve History Entry ===  
Retrieved: echo Hello  
  
=== Capacity Limit Test ===  
=== Command History ===  
1. ls  
2. echo Hello  
3. date  
4. whoami  
5. uname -a
```

```
=== History Consistency Test ===  
History buffer size maintained correctly  
  
=== Pipeline Structure ===  
Command 1: ls -l  
Command 2: wc -l  
  
=== Pipeline Execution ===  
3  
Pipeline execution completed  
  
=== Pipeline Layout Validation ===  
ls -l | wc -l  
Execution order maintained correctly
```